

Network Flow

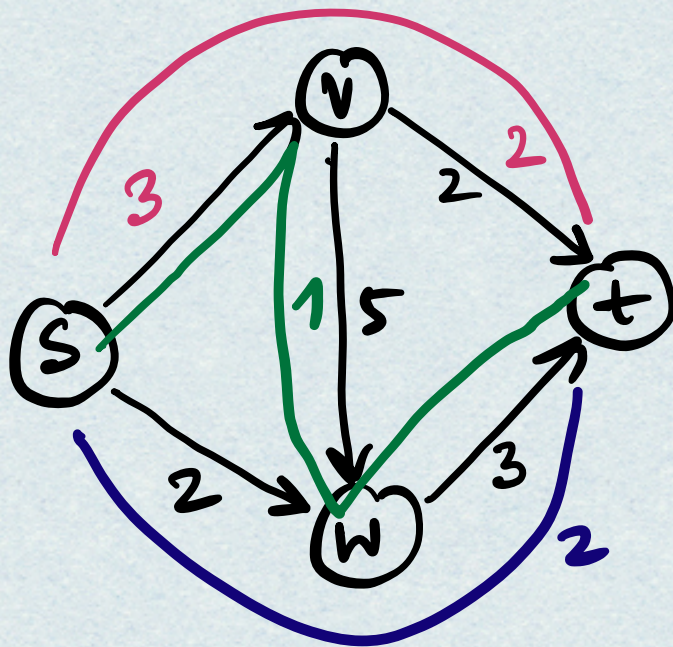
(Contd.)

Naive Greedy Algorithm

Recall- Greedy makes myopic & irrevocable decisions.

A-priori - it's unclear if such an algorithm exists.

Idea - Start with all-zero flow & greedily produce flows with even-higher value.



- Send more flow on some path $s \rightsquigarrow t$.

Naive Greedy Alg

Initialize $f_e = 0$, $\forall e \in E$

repeat

Search for all $s \rightsquigarrow t$ path P s.t $f_e < u_e$,
 for every $e \in P$.

// Takes $O(|E|)$ time
 using BFS / DFS

if no such path then
Halt with current
flow $\{f_e\}_{e \in E}$

else

$$\text{let } \Delta = \min_{e \in P} \underbrace{(u_e - f_e)}_{\text{room in } e}$$

$\underbrace{\hspace{10em}}_{\text{room in } P.}$

for all edges $e \in P$ do
increase f_e by Δ .

Note - Graph path needs to
determine whether or not
there is a $s \sim t$ path in
the subgraph of edges e with
 $f_e < u_e$.

This can be done by BFS/DFS.

- But there can be many such paths.

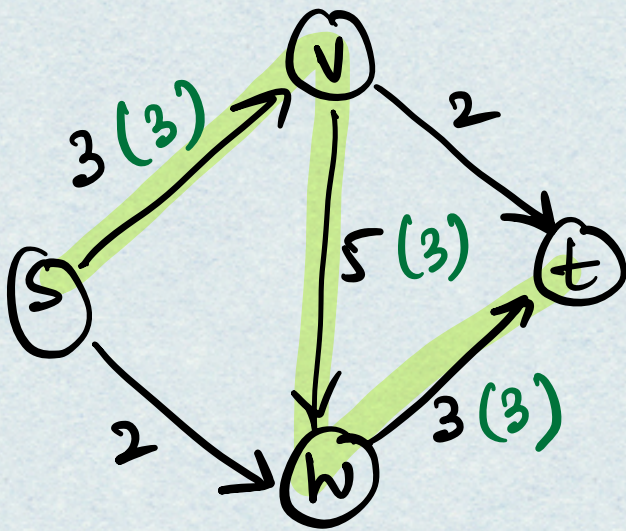
Which one to take..

For now, let's just take arbitrarily.

↓
push as much flow possible on that path.

Subject to the

Capacity Constraints.



$$\Delta = \min\{3, 5, 3\} \\ = 3$$

But it
Saturates

(s, v) 2
 (w, t)

Hence, Greedy terminates

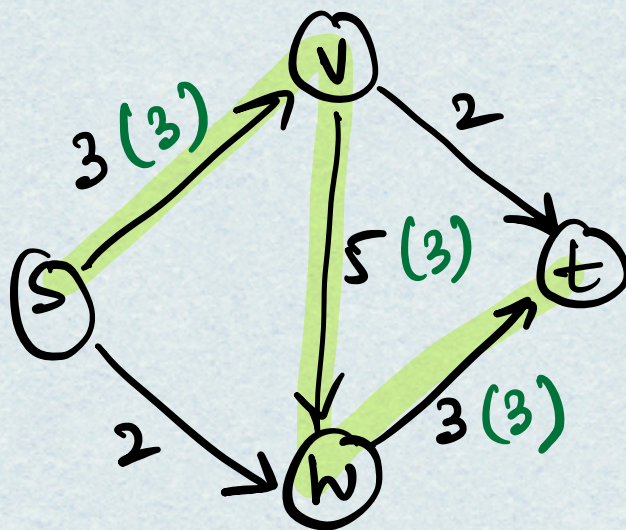
But we know from the
Example that we can
transmit 5 Unit.

Let's try something else

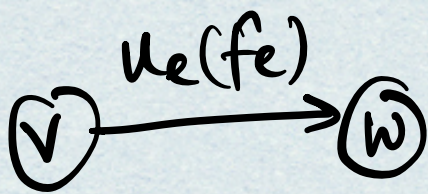
Residual Graphs

Can we extend "Greedy" but allow some "undo" operation

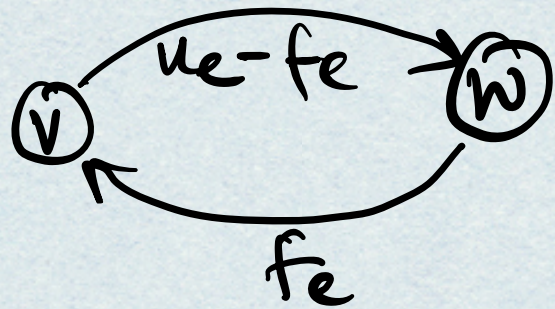
For instance,



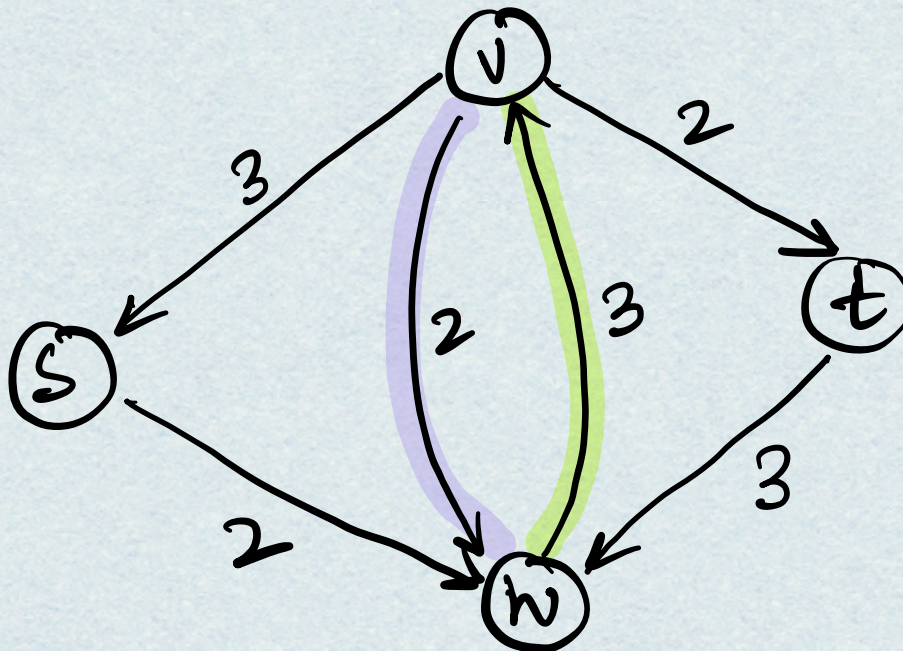
What if we
route (2)
unit in
 (s, w) &
then (2)
unit backward
 (v, w) &
finally (2) unit
on (v, t)



Original edge
Capacity &
Flow



Resultant edges
in residual
network.



Residual network Flow.

Formally specify the allowable "undo" operations

Residual Network

Given a graph G & a flow f in it.

We form a new flow network G_f that has

Same vertex set V as G .

$\forall e \in E$, $\exists$ two edges in E_f .

Flow assignment

An edge $e = (v, w)$ that
carries flow f_e & has
capacity u_e

Spawns a forward edge
 (u, v) of G_f with capacity
 $u_e - f_e$

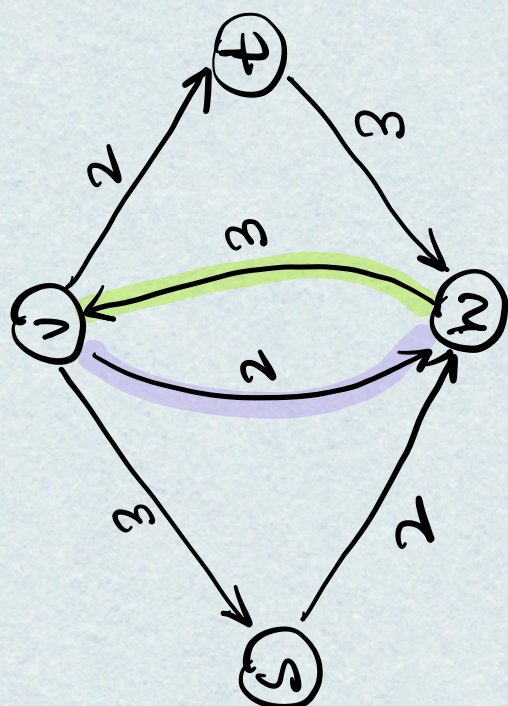
& a backward edge

(w, v) of G_f with capacity
 f_e

* This is the amount that
can be "undone".



Resultant edges
in residual
network.



(See G_x .)

Observe -

$s \sim t$ paths
with $f_e < u_e$
for all edges
as searched for
by naive greedy

Correspond to
the Special Case
of $s \sim t$ paths
of G_f that
comprise only
forward edges.

Ford - Fulkerson Alg.

let's run Greedy in the
Current residual network.

Initialize $f_e = 0 \quad \forall e \in E$.

repeat

Search for an $s \sim t$ path P
in the Current residual graph
 G_f s.t

// Takes $O(|E|)$ time
using BFS/DFS.

if no such path then
Halt with Current
flow $\{f_e\}_{e \in E}$.

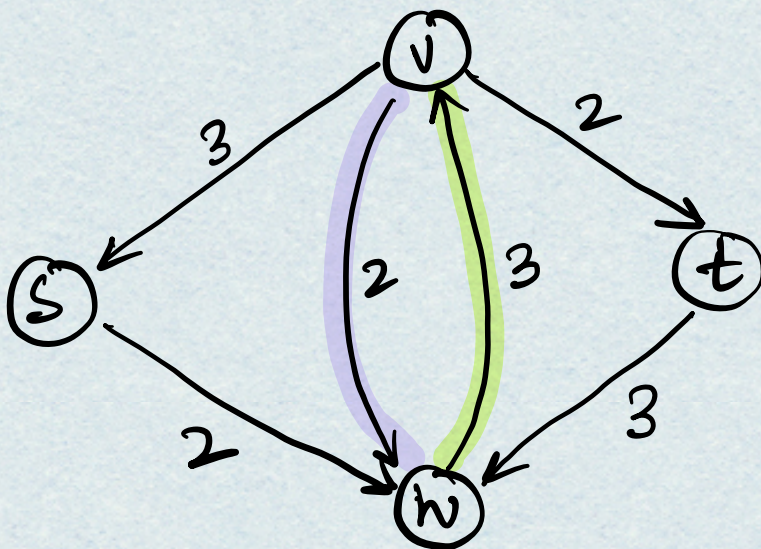
else

let $\Delta = \min_{e \in P} (e\text{'s residual
Capacity in
 } G_f)$

// Augment the flow f
using the path P .

for all edges e of G whose
corresponding forward edge
is in P do
increase f_e by Δ .

for all edges e of G
 whose corresponding reverse
 edges is in P do.
 decrease f_e by Δ .



Ford-Fulkerson will
 augment flow by units along
 with $s \rightarrow w \rightarrow v \rightarrow t$

Termination

Claim:- Ford-Fulkerson alg.
will eventually terminate with
a feasible flow.

- Follows from two
invariants. (Prove by induction
on # of iterations)

First,
the ALG. maintains the
invariant $\{f_e\}_{e \in E}$ is a flow.

This is clearly true
initially.

Parameter Δ is defined
so that no flow f_e
becomes negative or
exceeds the capacity u_e .

For Conservation Constraints

Consider a vertex v .

If v is not on the
augmenting path P in
 G_f , then flow into &

out of v remains

Same -

If v is on P
with edges (x, v) &
 (v, w) belonging to P

→ Cases.

If both are forward edges
then flow augmentation
increases both flow into
& flow out of v increase
by Δ .

→ If both are reverse edges, then both the flow into & flow out of v , decrease by v .

Bottomline

In all four cases, the flow in & flow out change by same amount so Conservation Constraint is preserved.

Second invariant

Ford-Fulkerson Alg maintains the property that every flow amount f_e is an integer.

(Recall, we assumed the edge Capacity u_e is an integer).

Inductively,

→ All residual Capacities are also integral.

→ So, the parameter Δ is also integral.

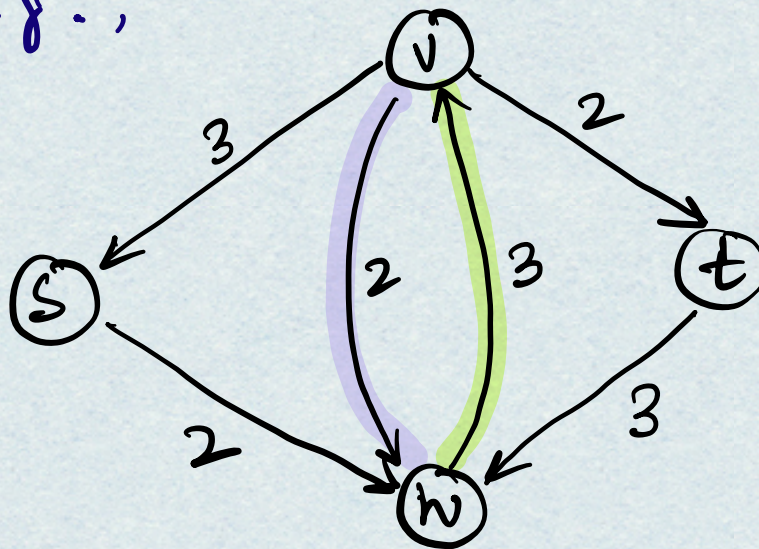
→ So, the flow stays integral.

Every iteration of

Ford-Fulkerson Alg.

it maintains the property that
every flow amount f_e is
an integer.

See, e.g.,



The second invariant
implies that $\Delta \geq 1$ in
each iteration of the
Ford-Fulkerson Alg.

Since only finite amount
of flow can escape the
Source vertex,

Ford-Fulkerson halts.

+

By the invariant -I it halts
with a feasible flow.

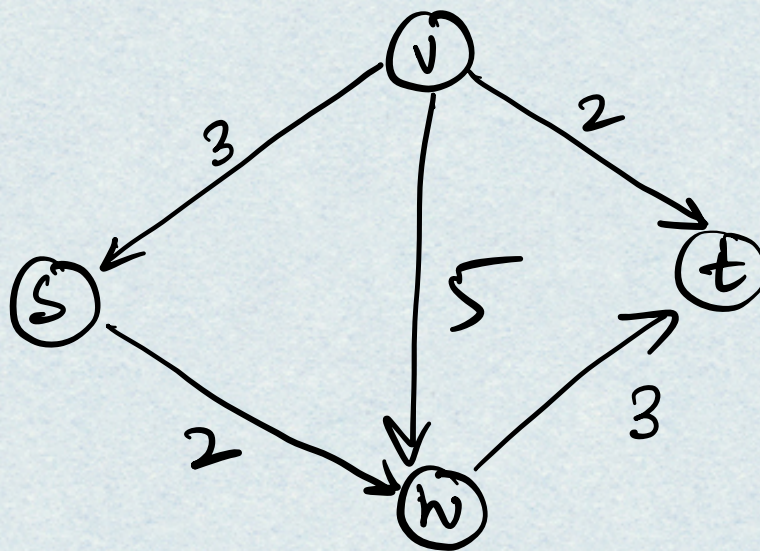
All of these apply usual.
"Greedy":

Q. How do we know that
Ford-Fulkerson won't
terminate with Non-Max flow?

Optimality

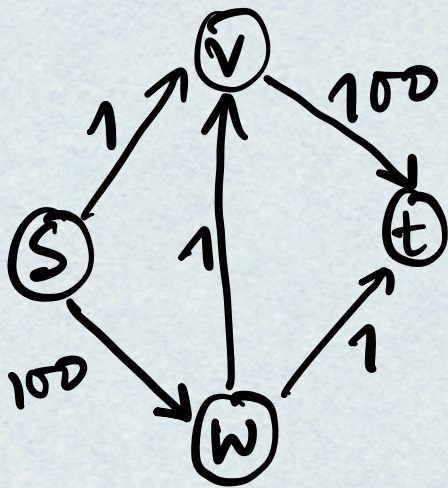
How do we know when
we are done?

In other words - given a flow,
how do we know it's a
Max flow?

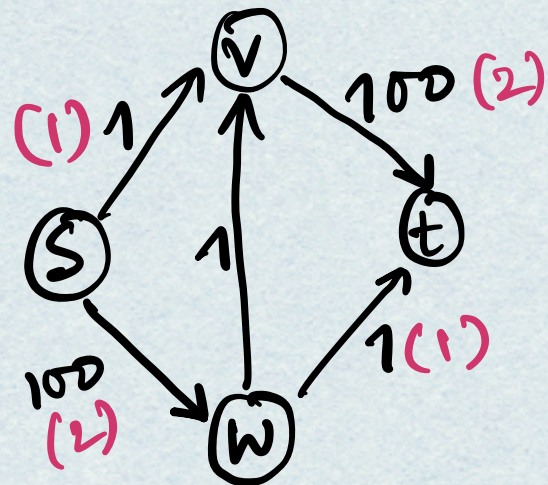


See here it's clear
 5. Since (v, w) is 5
 & also from
 s, max out flow = 5

But there can be
 higher value



2/p Network



Alleged max
flow of value
3.

let's define a paradigm
&
many computational
problem can be solved
by this paradigm.

Two-Step paradigm

1. Identify "optimality Conditions" for the problem.

This step is structural, & not necessarily algorithmic

(Remark: optimality conditions vary with the problem. but, often quite intuitive)

2. Design an algorithm that terminates with the optimality conditions satisfied.

Such an algo. is necessarily correct.

How do apply this two-step paradigm for Max. Flow.

Claim (optimality condition for Max Flow)

If f is a flow in G s.t. the residual network G_f has no $s \rightsquigarrow t$ path, then f is a maximum flow.

This claim implements the first step of the paradigm

The Ford-Fulkerson Alg
which can only terminate
with optimality condition
satisfied,
already provides a solⁿ.
to the second step.

we conclude:-

Corollary :- The Ford-Fulkerson Alg
is guaranteed to
terminate with a maximum
flow.